

RUST

Надёжность без отказа от производительности

БЕЗОПАСНОСТЬ · СКОРОСТЬ · КОНТРОЛЬ

```
fn main() {  
    println!("Hello, Rust!");  
}
```

Как запустить программу на Rust?

Локальное окружение

- rustup — установка toolchain
- cargo new project
- код → src/main.rs
- cargo run

Rust Playground

- ничего устанавливать не нужно
- код вводится в браузере
- компиляция — на удалённом сервере
- удобно для небольших примеров

Docker

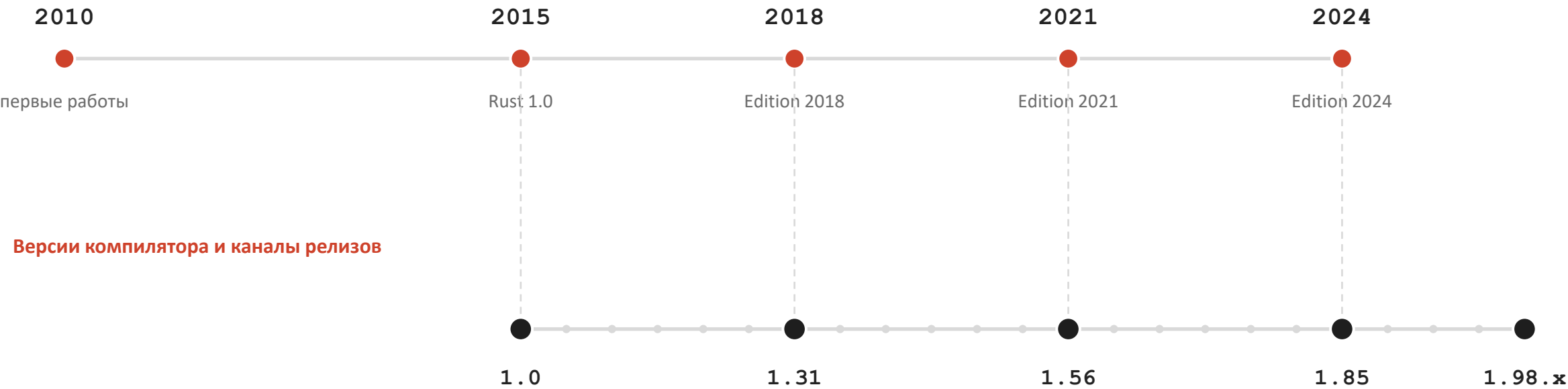
- запускаем образ с Rust
`docker run -it --rm rust`
- внутри контейнера
`cargo new`
- сборка и запуск — тоже через Cargo
`cargo run`
- компилятор изолирован в контейнере

```
$ rustup default stable → $ cargo new hello → $ cargo run
```

Во всех трёх случаях происходит одно и то же: `rustc` компилирует исходный код в машинный код — различается только окружение, в котором идёт компиляция.

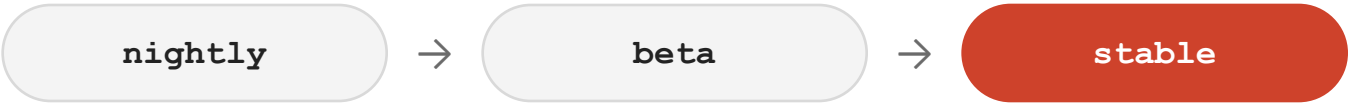
От Mozilla Research до самостоятельного языка

Редакции (editions)



Версии компилятора и каналы релизов

Новая версия компилятора выходит примерно каждые 6 недель — независимо от выхода редакций.



Три канала релизов обновляются параллельно, независимо друг от друга

Как исполняется программа Rust?



Rust ≠ интерпретируемый язык

Rust ≠ язык на JVM

Ahead-of-time compilation: программа компилируется заранее, без виртуальной машины при обычном запуске.

Какие идеи лежат в основе Rust?

Rust

- └─ системное программирование
- └─ контроль над ресурсами
- └─ memory safety
- └─ zero-cost abstractions
- └─ строгая статическая типизация
- └─ безопасность → на этапе компиляции

Если безопасность можно проверить компилятором, зачем проверять её во время выполнения?

Отсюда — ownership, borrowing, строгая система типов и zero-cost abstractions (подробно позже).

Какой Rust по своей парадигме?

Rust — мультипарадигменный язык

— **императивный**

переменные, условия, циклы

— **функциональный**

функции как значения, closures, итераторы, pattern matching

— **объектно-ориентированные черты**

без наследования классов

Как выглядит программа на Rust

```
1 fn main() {  
2     let name = "Rust";  
3  
4     println!("Hello, {name}!");  
5 }
```

`println!` — макрос, не функция

`main()`

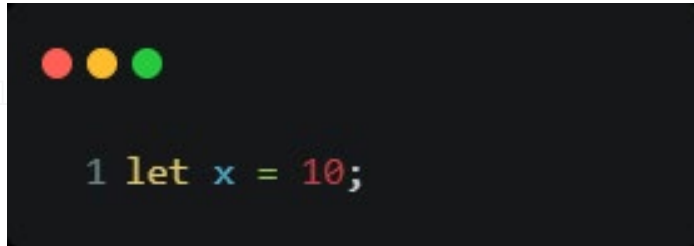


переменная



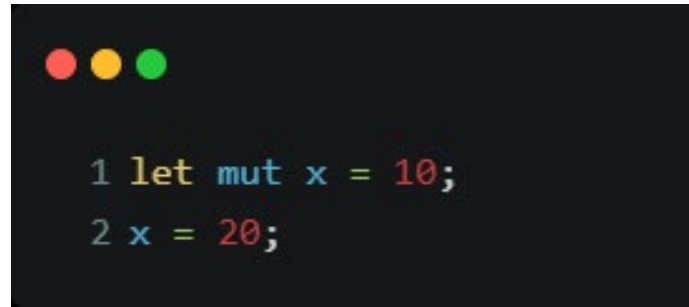
ВЫВОД

let, mut и shadowing



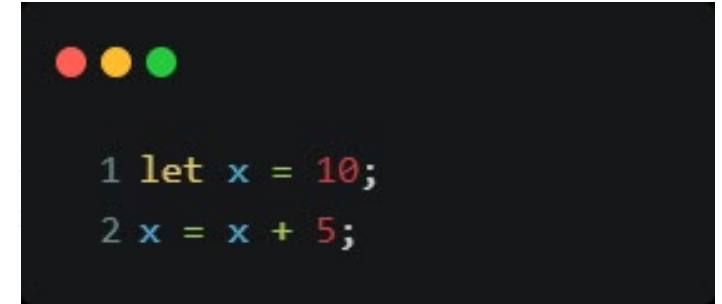
```
1 let x = 10;
```

неизменяемая



```
1 let mut x = 10;  
2 x = 20;
```

изменяемая



```
1 let x = 10;  
2 x = x + 5;
```

ошибка компиляции

неизменяемая → изменяемая → shadowing



```
let spaces = "  ";  
le  
1 let spaces = "  ";  
2 let spaces = spaces.len();
```

shadowing

Две разные переменные с одним именем — это не конфликт типов. Создаётся новая переменная, которая “затеняет”

Типы данных и область видимости



```
1 let x: i32 = 10;
2 let pi: f64 = 3.14;
3 let active: bool = true;
4 let c: char = 'A';
5
6 let pair: (i32, f64) = (10, 3.14);
7 let numbers: [i32; 3] = [1, 2, 3];
```

&T, &mut T — это ссылки. Java/C#-деление на primitive/reference type для Rust не подходит напрямую.



```
1 {
2     let y = 20;
3     println!("{y}");
4 }
5 // y здесь уже недоступна
```

Каждый блок { ... } — своя область видимости.

Для владеющих данных конец области видимости — точка, где Rust освобождает ресурсы.

Явное преобразование типов



```
1 let x: i32 = 10;  
2 let y: f64 = x + 1.0;
```

✗ Ошибка компиляции



```
1 let x: i32 = 10;  
2 let y: f64 = x as f64;
```

✓ Явное преобразование через as

Потенциально неоднозначные действия лучше делать явными, чем скрывать за неявными преобразованиями.

Условия и циклы

```
1 if condition1 {
2     ...
3 } else if condition2{
4     ...
5 }else{
6     ...
7 }
```

if / else — условие

```
1 while condition {
2     ...
3 }
```

while — пока верно условие

```
1 loop {
2     ...
3 }
```

loop — бесконечный, до break

```
1 for i in 0..5 {
2     println!("{i}");
3 }
```

for — перебор последовательности

```
1 for x in collection {
2     println!("{x}");
3 }
```

for — перебор коллекции

Факт:

for x in collection / in collection.into_iter() — перебираем элементы, получая их во владение.

for x in collection.iter() — перебираем элементы по ссылке, не забирая владение.

for x in collection.iter_mut() — перебираем элементы по изменяемой ссылке и можем изменять их.

Условия и циклы - выражения

```
1 let result = if condition {
2     10
3 } else {
4     20
5 };
```

if / else — условие

```
1 let result = loop {
2     break 42;
3 };
```

loop — бесконечный, до break

В Rust блоки и управляющие конструкции могут быть выражениями.

```
1 let result = while x < 10 {
2     x += 1;
3 };
```

For/while -> () – возвращают пустой кортеж

Функции и рекурсия

```

1 fn add(a: i32, b: i32) -> i32 {
2     a + b
3 }

```

```
let result = add(2, 3);
```

```

1 let result = add(2, 3);

```

```

1 fn factorial(n: u32) -> u32 {
2     if n == 0 {
3         1
4     } else {
5         n * factorial(n - 1)
6     }
7 }

```

рекурсия

Тип возвращаемого значения — после ->.
Последнее выражение без ; становится результатом функции.

Обработка ошибок

```
panic!("Something went wrong!");
```

```
assert!(condition);
```

```
assert_eq!(a, b);
```

panic!

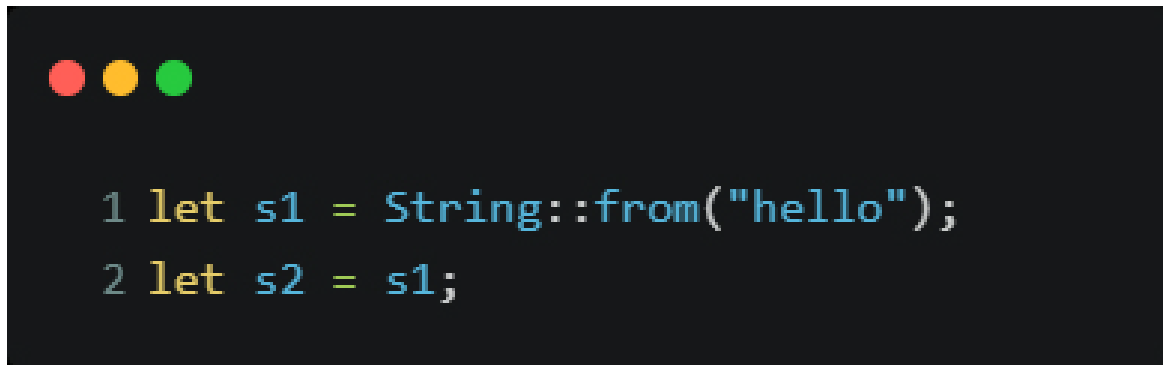
→ невозстановимая ошибка

Result<T, E>

→ ожидаемая ошибка

Нет try/catch и иерархии исключений. Result, Option и оператор ? — тема следующей презентации.

Ownership: как Rust управляет значениями



```
1 let s1 = String::from("hello");  
2 let s2 = s1;
```

s1 —X

s2 —→ String("hello")

move: владение перешло от s1 к s2 — использовать s1 после этого нельзя.

1

Каждое значение имеет ровно одного владельца.

2

Владение можно передать (move) — тогда прежний владелец становится недействительным.

3

Когда владелец выходит из области видимости — значение уничтожается.

4

В этот момент Rust автоматически вызывает деструктор (impl Drop) — в том числе для вложенных полей.

Так Rust исключает ситуацию, когда два владельца одновременно пытаются освободить один и тот же ресурс.

C++ · RAII

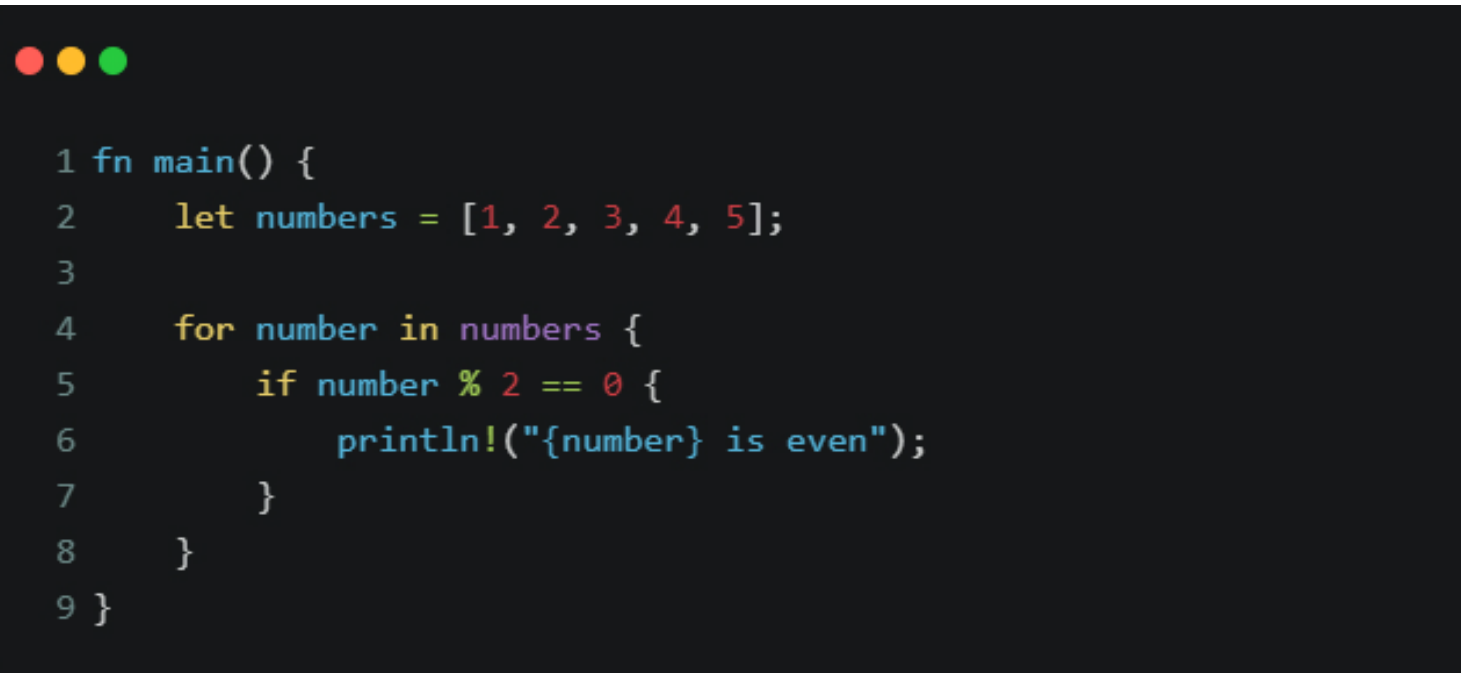
Ресурс привязан к времени жизни объекта.
Когда объект уничтожается — его деструктор освобождает ресурс.

Rust · Ownership + Drop

значение → владелец → выход из scope
→ `drop()` → освобождение ресурсов

Осторожно: не каждый тип реализует RAII/Drop — например, `i32` — Сору-тип без пользовательского деструктора.

Маленькая программа целиком



```
1 fn main() {  
2     let numbers = [1, 2, 3, 4, 5];  
3  
4     for number in numbers {  
5         if number % 2 == 0 {  
6             println!("{number} is even");  
7         }  
8     }  
9 }
```

функция main



массив



цикл for



условие if



оператор %



ВЫВОД

Синтаксис выглядит привычно — и это специально: он почти не показывает главных особенностей Rust.

Синтаксис знаком. Модель памяти — нет.

Где начинается настоящий Rust?

Ответ — не в `if`, `for` или `fn`, а в модели данных и памяти: кто владеет значением, кто может его использовать, когда освобождается память.

